SARD

Sistema Agéntico de Respuesta y Resiliencia ante Desastres

Un reporte de emergencia que sobrevive a la caída de la red y de un nodo, y llega priorizado a quien decide.

Hackathon Expo ASI 2026 · Universidad Nacional de Colombia, Sede Manizales

Prototipo con datos sintéticos, probado en emuladores de Android y iOS.



Dos momentos críticos, dos riesgos

Primeros minutos

Pánico, red saturada y teléfonos con menos del 20 % de batería. El riesgo: que el reporte se pierda.

Pasadas 6 horas

El 95 % de los teléfonos está apagado y los despachadores trabajan a oscuras. El riesgo: clasificar mal.

El problema ya no es obtener datos, sino decidir con ellos. Cuatro decisiones:

Urgencia
¿Qué no puede esperar?

Confiabilidad
¿Qué tan seguro es?

Destino
¿A dónde se envía?

Persistencia
¿Sobrevive si el sistema cae?

Fuente: enunciado de la Hackathon SARD, secciones 1 y 3

El recorrido completo, en ocho pasos

1 · Captura

Reporte

SOS en un toque, en la app Flutter.

2 · Captura

Guardado local

SQLite durable; se confirma tras el commit.

3 · Transporte

Envío intermitente

Trama de 33 B, ACK selectivo y reintentos.

4 · Decisión

Agentes

Seis agentes; triaje por reglas y, opcional, modelo.

5 · Decisión

Priorización

P1 a P4, con la evidencia citada.

6 · Decisión

Tablero

Lista corta y aprobación humana.

7 · Resiliencia

Pérdida del nodo

kill -9 con la ráfaga a medias.

8 · Resiliencia

Recuperación

Réplica entre nodos; huellas idénticas.

Un mismo reporte recorre los ocho pasos, y en el 7 un nodo muere de verdad.

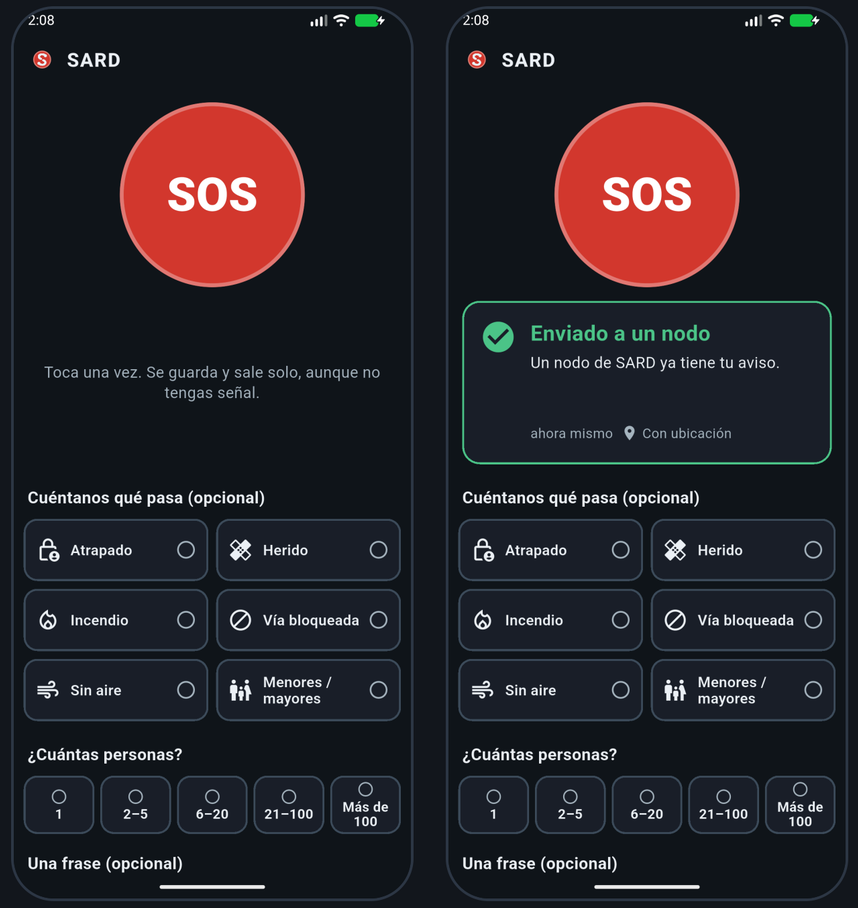
Recorrido mínimo exigido por el enunciado (figura 6) · datos sintéticos y canales simulados

Una persona asustada hace una sola cosa

33 B por trama de SOS. Los detalles viajan aparte, como revisiones del mismo reporte.

6 de 6 golpes de kill -9 sin perder un SOS dado por guardado (T01).

≈ 1 s para salir sola al volver la señal, medido en el emulador de Android.



Capturas reales del emulador de Android · docs/resultados/b5.md §1 (T01) y b2-dispositivo.md

Seis agentes, sin nodo central

Agente	Qué hace	Su límite
1 · Recepción	Valida y normaliza el reporte	No inventa la ubicación
2 · Triaje	Estima urgencia y cita la evidencia	No baja la urgencia por falta de datos
3 · Correlación	Propone posibles duplicados	Conserva el vínculo con cada original
4 · Comunicación	Decide qué se envía primero	Lo de baja prioridad no desaparece
5 · Resiliencia	Vigila los nodos y reanuda	Sin modelo de lenguaje
6 · Coordinación	Recomienda una acción	No despacha: exige aprobación humana

> Por omisión funcionan con reglas. El modelo de lenguaje es opcional (apagado por omisión) y se midió contra la API real: lámina siguiente.

Fichas completas en el PDF de arquitectura, capítulo 6 · docs/resultados/b3.md

El modelo real captura más P1, con un coste

Recall de P1

85,4 → 96,0 %

en 750 reportes con texto, con Sonnet 5.5. Con Opus 5.5: 98,5 %.

Falsos P1

0 → 10

de 750: todos eran P2 subidos un nivel. Con Opus: 24, a doble coste.

Latencia y coste

p95 4,4 s

0 de 749 pasaron de los 8 s del nodo. 0,0024 USD por reporte.

Con textos que dan instrucciones maliciosas, 12 de 12 cumplen: el modelo nunca baja la urgencia y no puede despachar.

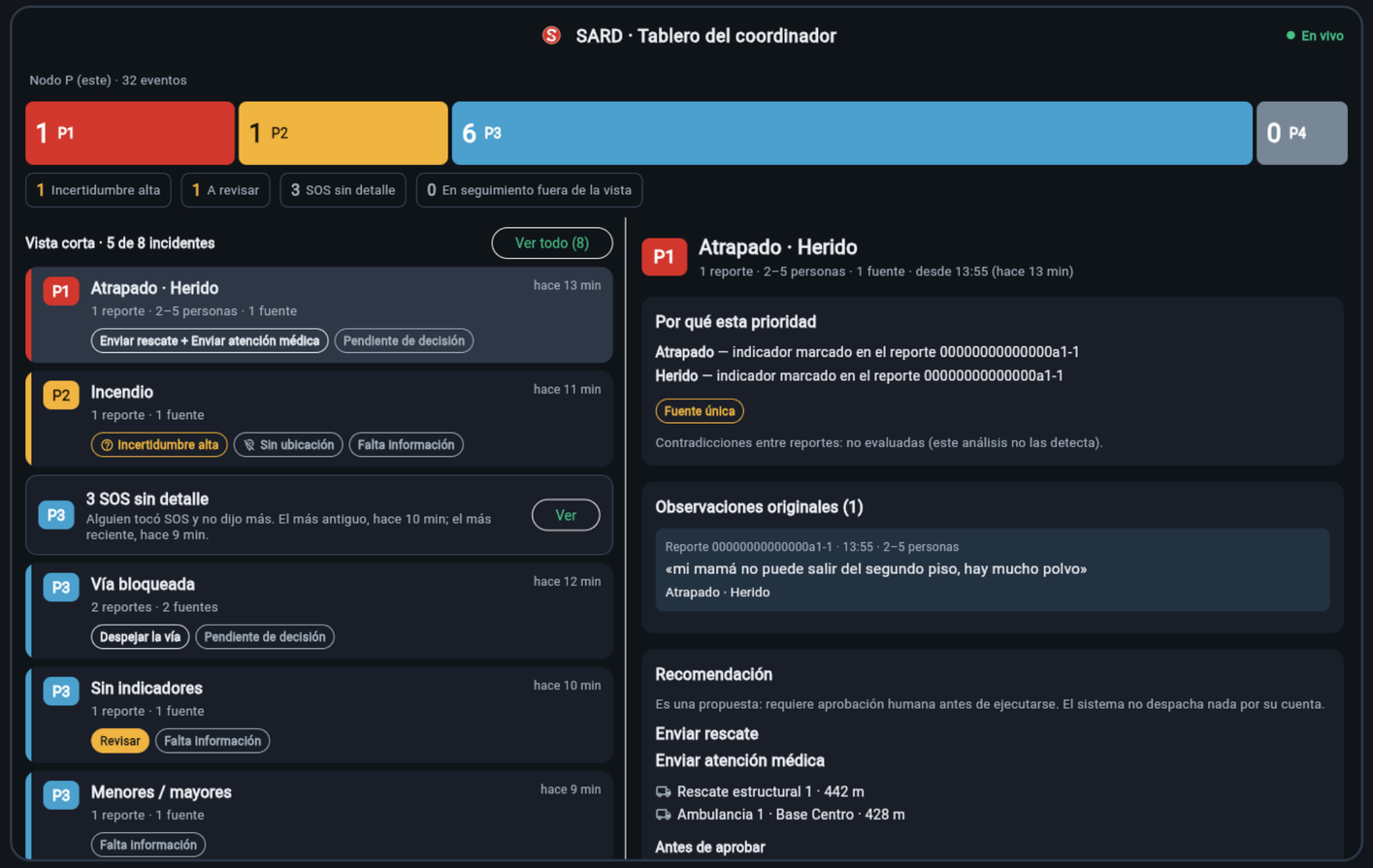
Datos sintéticos, una corrida, solo reportes con texto: T06 completo sigue sin cumplirse (≈ 85 % estimado) · docs/resultados/modelo-real.md

El tablero ordena y explica

0 acciones o recursos inventados llegan a la persona (T13).

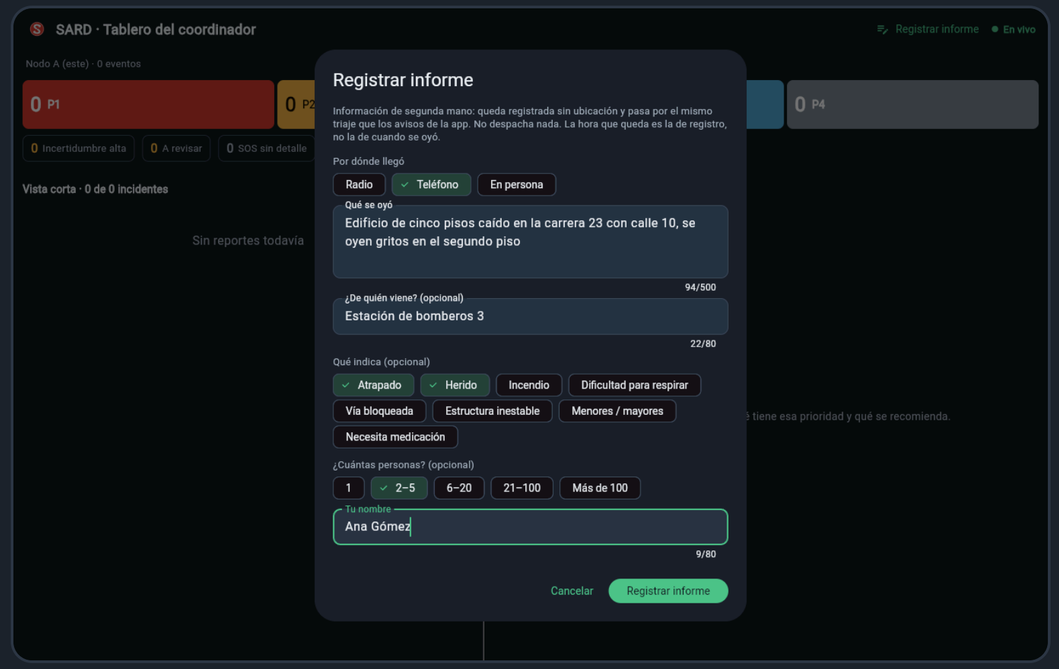
0 aprobaciones automáticas: toda propuesta espera a una persona.

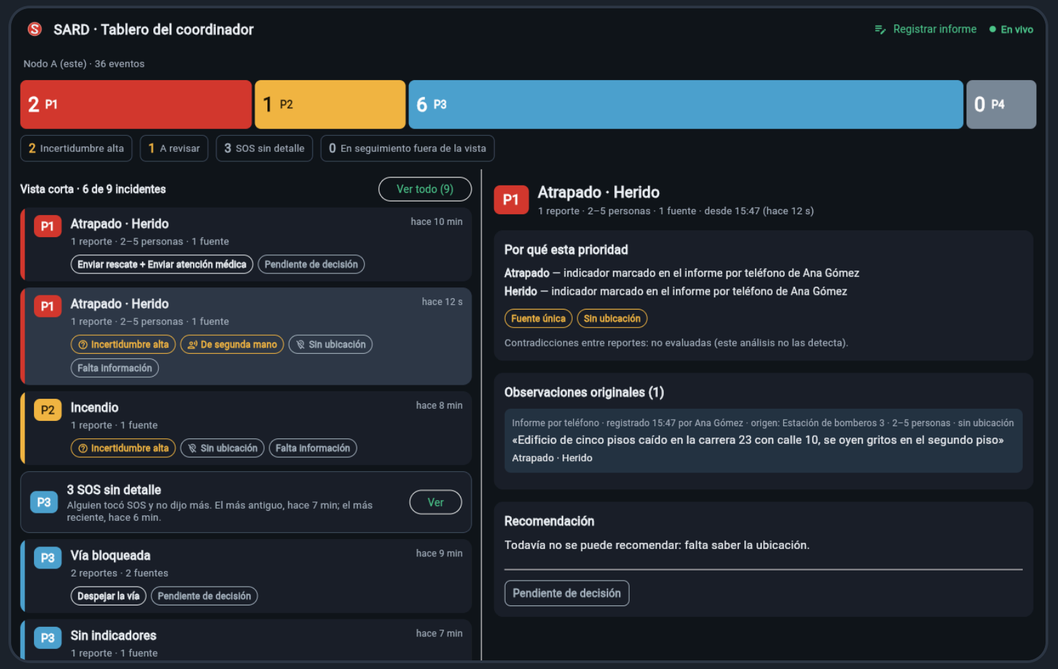
0 P1 fuera de la vista inicial, en 5 semillas nuevas (T14).



Captura real del tablero con datos sintéticos · docs/resultados/b5.md §1 (T13, T14)

El despachador registra lo que oyó





De segunda mano
Una fuente, confianza 0,2.

Sin ubicación
No se inventa: no hay recomendación.

Un rumor no se amplifica
El mismo texto de varios orígenes es una fuente.

Capturas reales del tablero · docs/resultados/despachador.md (T16) · no resuelve el Reto 4 ni se probó con despachadores reales

Mato un nodo en pleno trabajo

Ráfaga de 26 reportes hacia el nodo A; kill -9 a mitad del procesamiento.

8 de 26

los acusó A antes de morir; los otros 18 pasaron a B. Todo lo acusado seguía en su disco al reiniciar (T09).

2,3–3,7 s

hasta que el nodo vuelve a responder (RTO), con dos nodos reales.

3,1–3,6 s

para converger tras reconectar: las huellas de los dos nodos son idénticas (T10).

Límite: las particiones probadas duraron 5–20 s por configuración, no 10 minutos ni una falla de red real.

tool/demo.sh · docs/resultados/b5.md §1 (T09 y T10)

10 000 reportes: de 516 s a 10 s



Latencia del ACK, p95
1 742 → 32 ms

Primer P1 visible
4,1 → 2,6 s

Llegaron a la cola
10 002 de 10 002

La primera medición mostró un defecto: por cada trama el nodo recorría todos los reportes (trabajo proporcional al número de reportes, en la ruta del ACK). Se corrigió y se midió de nuevo, con 16 clientes contra el proceso real.

dart run packages/sim/bin/burst.dart --reports 10000 · docs/resultados/b5.md §1 y §3

Cuando algo falla, el sistema sigue

El modelo se cuelga

El ACK p95 pasa de 9 a 17 ms y todas las tramas se acusan. Antes, con 1 024 descriptores, el nodo moría: se corrigió con un tope de consultas.

El modelo era un servidor falso.

Dos coordinadores se contradicen

Aislados, uno aprueba y otro rechaza. Se conservan las dos decisiones, manda el rechazo y la discrepancia queda a la vista.

Misma semilla, misma huella

La demostración se repite y da el mismo SHA-256 de lo que ve el coordinador, sin relojes ni identificadores internos (T15).

Cada falla se provoca a propósito y se mide con procesos reales del nodo.

docs/resultados/b5.md §1 y §4 · t12.md

Lo que no cumplimos

78,5 %

de recall de P1 por reglas (≈ 85 % con el modelo, estimado), frente al 95 % pedido (T06).

0,43 %

de los pares verdaderos se agrupan: las 5 fuentes independientes no forman un grupo.

1 corrida

con la API real y datos sintéticos; T12 sigue con un servidor falso.

0

coordinadores, teléfonos o radios reales en las pruebas: emuladores y datos sintéticos.

5–20 s

duraron las particiones probadas; el enunciado habla de 10 minutos.

Anónima

la aprobación: no se autentica a quien decide; el código de acceso no lo identifica.

Falta además el mapa de cobertura y la ubicación en los informes del despachador · docs/resultados/

Lo que sigue, por orden de valor

1. **Agrupar de verdad**
   Similitud semántica con el modelo real, para que las fuentes independientes formen un grupo (Reto 4).

2. **Probar con coordinadores**
   Medir si el tablero reduce la carga de quien decide; hoy solo medimos elementos a revisar.

3. **Mejorar el canal del despachador**
   Hoy un informe llega sin ubicación (el agente 6 no recomienda) y un mensaje cortado no se marca (Escenario 2).

4. **Cobertura y privacidad**
   Un mapa que avise de que el silencio no es seguridad, y redactar datos personales antes de llegar al modelo.

Orden propuesto por quien desarrolló el prototipo · README.md, sección Pendiente

Se puede reproducir

```
dart pub get
bash tool/test_all.sh # 1 330 pruebas, ≈ 6 min
PART=4 bash tool/demo.sh 42 # partición, kill -9, misma huella
dart run packages/sim/bin/burst.dart --reports 10000
```

Documentos

PDF de arquitectura (50 págs.), fichas de agentes y protocolo.

Resultados

docs/resultados/: cada cifra, con el comando que la produce.

Datos

Sintéticos y reproducibles por semilla.

Lo que está dicho aquí está medido; lo que no, está en la lámina «Lo que no cumplimos».

README.md · docs/resultados/b5.md §8 lista cada comando